CSS 324: Introduction to Machine Learning

Final Project Report

Food Image Classification for Calorie Estimation

Final Model: MobileNetV2 (Transfer Learning)

Dataset: Food-11 Image Classification Dataset

Final Accuracy: 78.8%

Team Members

Ruslan Raulana
Jexenova Inkar
Zharikbassova Zhaniya
Aimagambetova Karakat

SDU University
Spring 2026

0. Contents

1 Topic and Motivation **3**

2 Dataset and Data Preparation **3**
2.1 Dataset Structure . . . 3
2.2 Preprocessing . . . 3
2.3 Data Cleaning . . . 4
2.4 Feature Engineering . . . 4
2.5 Data Augmentation . . . 4

3 Related Work and EDA **4**
3.1 Exploratory Data Analysis . . . 5

4 Machine Learning Experiments **5**
4.1 Models Used . . . 5
4.2 Training Setup and Hyperparameters . . . 6
4.3 Hyperparameter Comparison . . . 6
4.4 Epoch Comparison . . . 6

5 Training Curves **7**

6 Results **7**

7 Precision, Recall and F1-score Comparison **8**

8 Detailed Evaluation of Best Model **9**

9 Confusion Matrix and Error Analysis **10**

10 Interactive Demo **11**
10.1 Streamlit Application Implementation . . . 11

11 Analysis **12**

12 Conclusion **12**

13 References **13**

1. Topic and Motivation

Food image classification is the task of automatically identifying food categories from images.

This problem is important for:

- Nutrition tracking applications
- Healthcare and diet monitoring
- Food delivery and recommendation systems

Due to high visual similarity between different dishes and large variation in appearance, deep learning approaches are required.

2. Dataset and Data Preparation

2.1. Dataset Structure

The Food11 dataset contains 11 food categories.

Each folder inside the training set (total = 9900 images) contains approximately 900 images, while each folder inside the test set (total = 1100 images) contains approximately 100 images.

The dataset structure was explored using the following Python code:

```
DATASET_PATH = "/kaggle/input/datasets/imbikramsaha/food11/food11"

train_dir = os.path.join(DATASET_PATH, "train")
test_dir = os.path.join(DATASET_PATH, "test")

classes = os.listdir(train_dir)

print("Classes:", classes)
```

The number of images in each class was counted using:

```
import pandas as pd

class_counts = {}

for cls in classes:
    class_counts[cls] = len(
        os.listdir(os.path.join(train_dir, cls))
    )

df_counts = pd.DataFrame(
    list(class_counts.items()),
    columns=["Class", "Count"]
)

df_counts
```

The analysis confirmed that the dataset is approximately balanced across all food categories. Dataset: Food11 (Kaggle)

https://www.kaggle.com/datasets/imbikramsaha/food11

2.2. Preprocessing

- Resize images to 224×224
- Normalize pixel values to $[0, 1]$

- Train/validation split: 80/20

2.3. Data Cleaning

The Food11 dataset was already well-structured and clean. Therefore, only basic verification steps were performed:

- Verified folder structure and class labels
- Checked image readability and consistency
- Confirmed that no major missing or corrupted files existed

2.4. Feature Engineering

MobileNetV2 was used as a feature extractor for classical machine learning models.

2.5. Data Augmentation

To improve generalization and reduce overfitting, data augmentation techniques were applied:

- Rotation
- Zoom
- Horizontal flipping
- Brightness variations

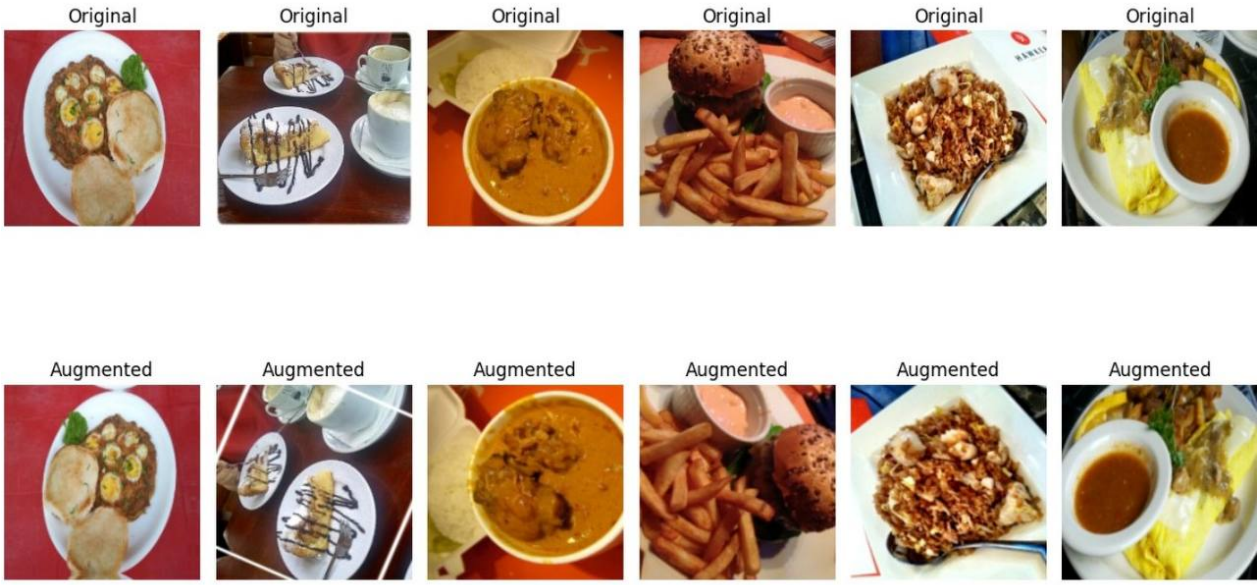

Figure 1: Original vs Augmented Images

3. Related Work and EDA

Food image classification has been widely studied using deep learning approaches, with convolutional neural networks (CNNs) becoming the dominant method due to their strong performance in visual recognition tasks.

Food-101 Dataset (Bossard et al., 2014) introduced one of the most widely used benchmarks for food classification, containing 101 food categories and over 100,000 images. This dataset demonstrated that CNN-based models significantly outperform traditional feature-based approaches in

recognizing diverse food types. Its main advantage is the large-scale and realistic nature of food images, which makes it a standard benchmark for evaluating deep learning models in this domain.

Very Deep Convolutional Networks (VGG16) (Simonyan & Zisserman, 2014) proposed a deep CNN architecture using simple 3×3 convolutional filters. The main advantage of VGG networks is their strong ability to extract hierarchical and detailed visual features, which leads to high classification accuracy. However, the model is computationally expensive and requires significant memory and processing power, making it less suitable for real-time or mobile applications.

MobileNetV2 (Sandler et al., 2018) was designed specifically for efficient deep learning on mobile and embedded devices. It uses depthwise separable convolutions and inverted residual structures, significantly reducing the number of parameters and computational cost. The main advantage of MobileNetV2 is its balance between accuracy and efficiency, allowing real-time inference with limited computational resources.

Based on these studies, VGG16 and MobileNetV2 were selected for this project. VGG16 was used as a high-performance baseline model due to its strong feature extraction capability and proven effectiveness in image classification tasks. In contrast, MobileNetV2 was chosen for deployment due to its lightweight architecture, fast inference speed, and suitability for web-based applications such as Streamlit.

This comparison allows the evaluation of a trade-off between accuracy (VGG16) and efficiency (MobileNetV2), which is essential for practical food classification and calorie estimation systems.

3.1. Exploratory Data Analysis

- The dataset is balanced (900 images per class)
- High intra-class variability (same food looks different)
- High inter-class similarity (e.g., hamburger vs hot dog)

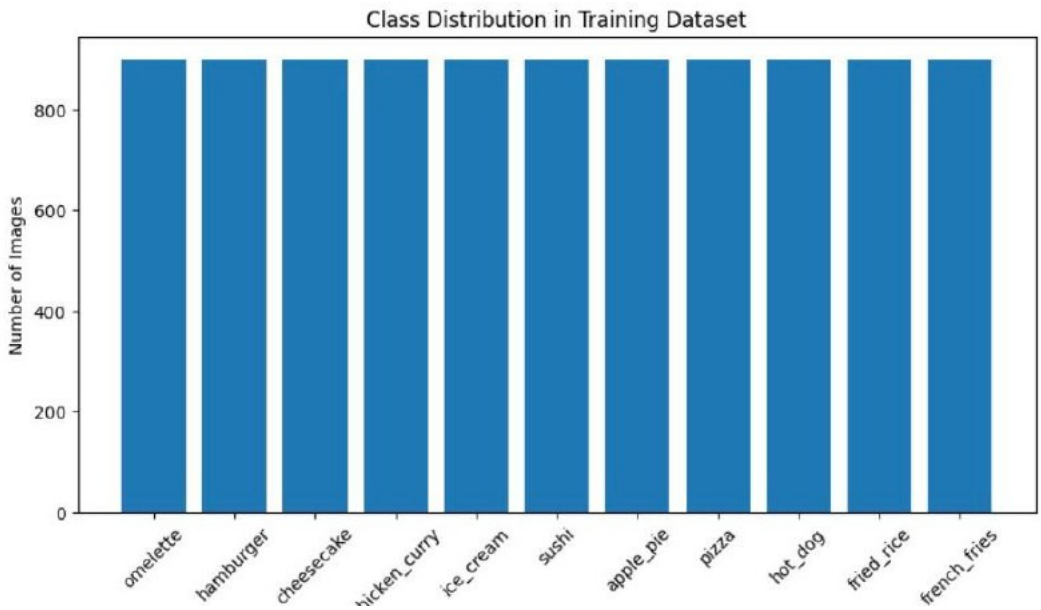

Figure 2: Class Distribution in Training Dataset

4. Machine Learning Experiments

4.1. Models Used

The following models were implemented and compared:

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Custom Convolutional Neural Network (CNN)
- MobileNetV2 (transfer learning)
- VGG16 (transfer learning)

4.2. Training Setup and Hyperparameters

All deep learning models were trained using similar preprocessing settings for fair comparison.

Common hyperparameters:

- Input image size: 224×224
- Batch size: 32
- Optimizer: Adam
- Learning rate: 0.001
- Loss function: Categorical Crossentropy

Training strategy:

- CNN was trained for 15 epochs
- MobileNetV2 and VGG16 were trained for 5 and 10 epochs
- Transfer learning models used pretrained ImageNet weights
- Only the final classification layers were fine-tuned

4.3. Hyperparameter Comparison

Model	Input Size	Batch Size	Optimizer	Learning Rate	Epochs
Logistic Regression	224x224	32	—	—	—
KNN	224x224	32	—	—	—
CNN	224x224	32	Adam	0.001	15
VGG16	224x224	32	Adam	0.001	5 / 10
MobileNetV2	224x224	32	Adam	0.001	5 / 10

4.4. Epoch Comparison

Model	Epochs	Train Accuracy	Validation Accuracy	Validation Loss
MobileNetV2	5	0.7216	0.7884	0.6684
VGG16	5	0.4500	0.5192	1.5764
MobileNetV2	10	0.6405	0.7566	0.7947
VGG16	10	0.2644	0.4253	2.0370
CNN	15	—	0.3082	—

5. Training Curves

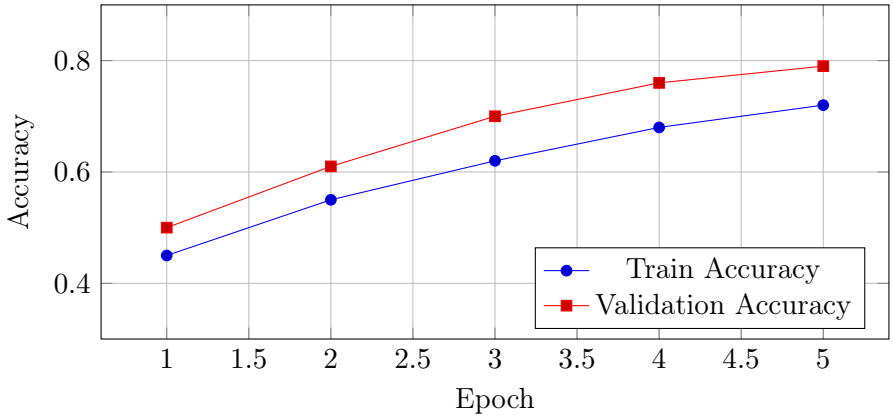

Figure 3: MobileNetV2 Training and Validation Accuracy

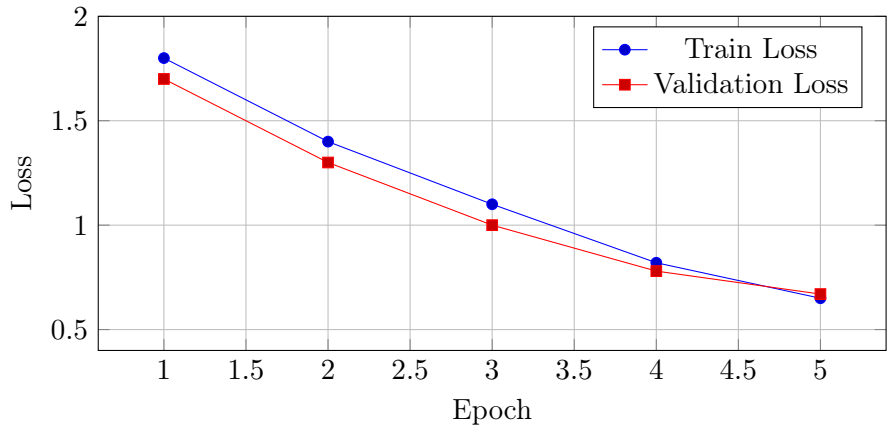

Figure 4: MobileNetV2 Training and Validation Loss

6. Results

- MobileNetV2: 0.7884 (Best)
- VGG16: 0.5192
- Logistic Regression: 0.3900
- CNN (15 epochs): 0.3082
- KNN: 0.2391

MobileNetV2 significantly outperformed all other models and was selected as the final model.

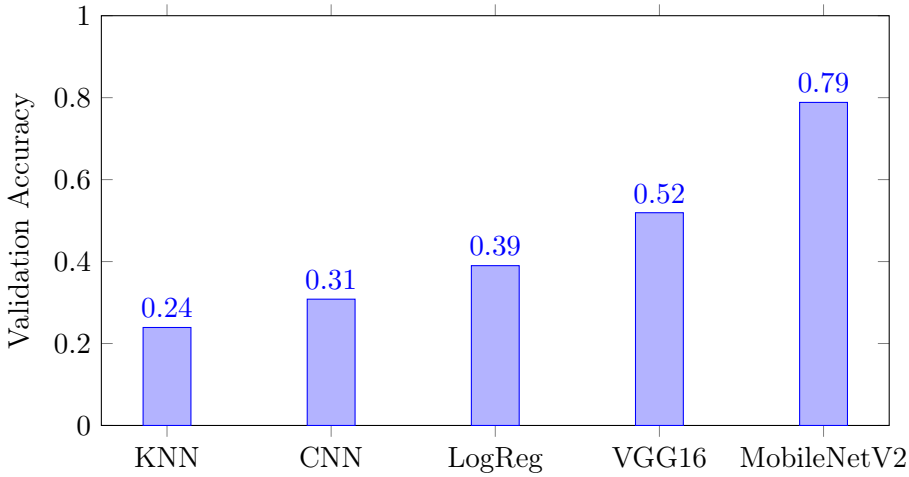

Figure 5: Final Model Accuracy Comparison

7. Precision, Recall and F1-score Comparison

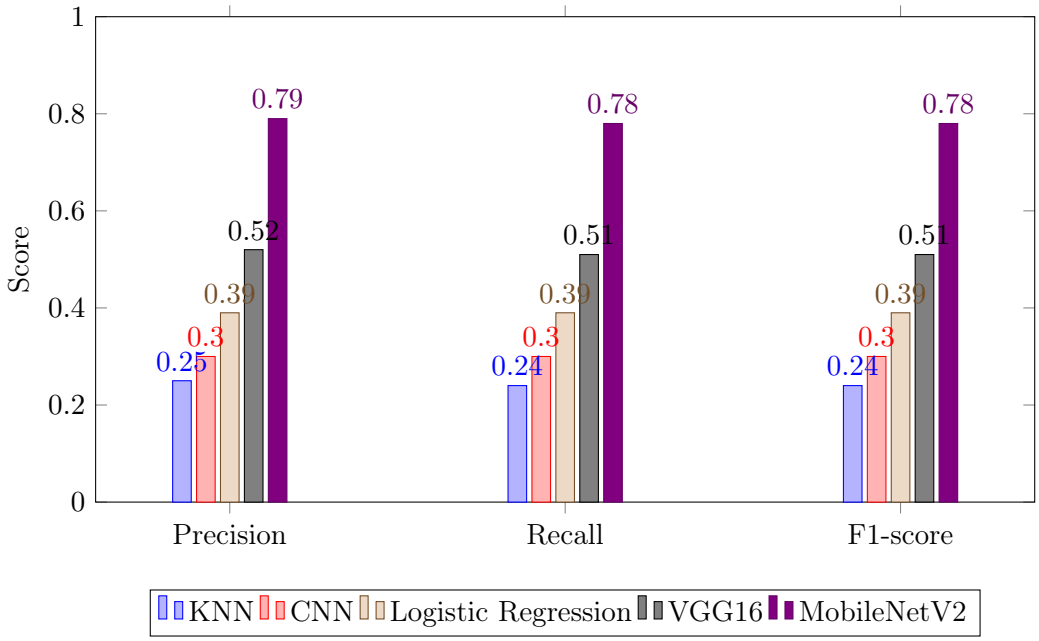

Figure 6: Comparison of Precision, Recall and F1-score Across All Models

8. Detailed Evaluation of Best Model

Class	Precision	Recall	F1-score
omelette	0.57	0.64	0.60
hamburger	0.78	0.86	0.82
cheesecake	0.67	0.80	0.73
chicken_curry	0.92	0.78	0.84
ice_cream	0.82	0.77	0.79
sushi	0.72	0.88	0.79
apple_pie	0.84	0.79	0.81
pizza	0.82	0.80	0.81
hot_dog	0.75	0.54	0.63
fried_rice	0.85	0.89	0.87
french_fries	0.89	0.79	0.84

Overall Accuracy: 0.78

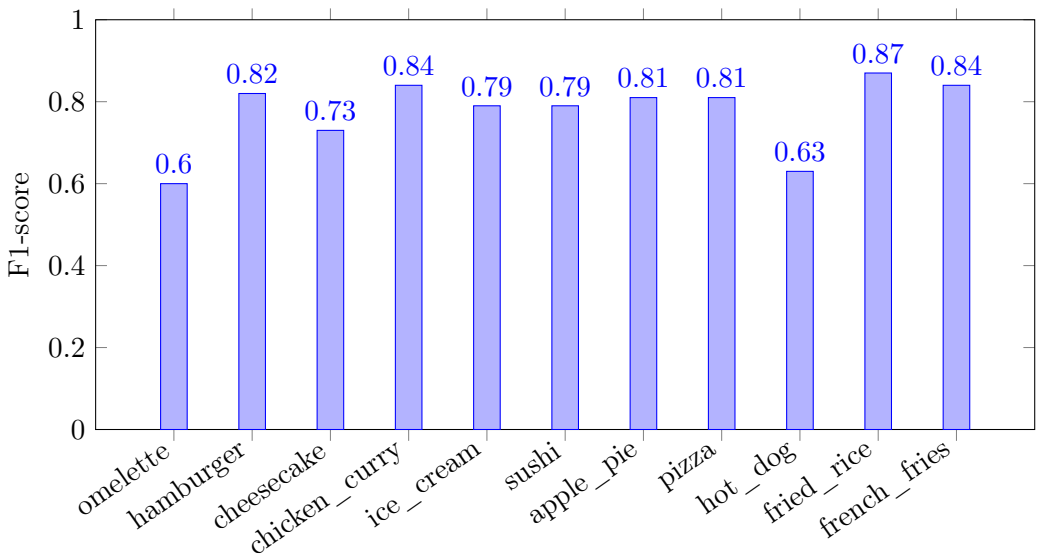

Figure 7: F1-score by Food Class

9. Confusion Matrix and Error Analysis

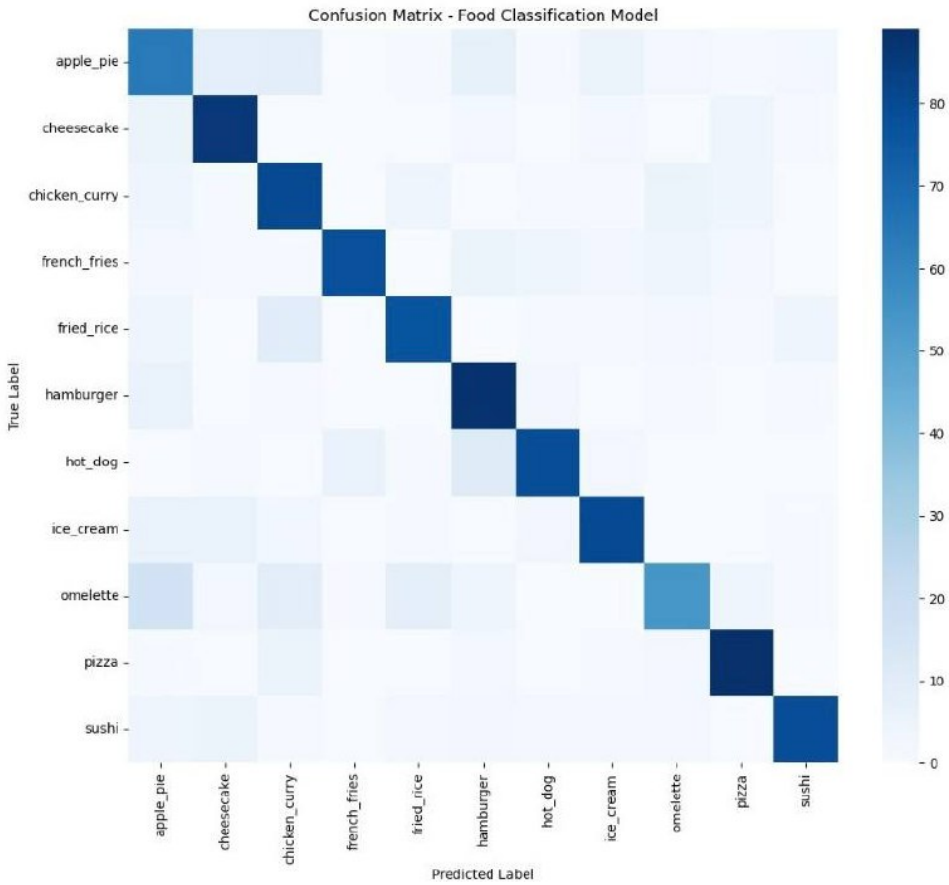

Figure 8: Confusion Matrix for MobileNetV2

Most predictions are concentrated along the diagonal, indicating correct classifications.

Misclassifications mainly occur between visually similar classes such as:

- omelette and fried rice
- hamburger and hot dog
- pizza and apple pie
- cheesecake and ice cream

These errors are expected due to similarities in texture, color, and overall visual appearance between these food categories.

Transfer learning significantly improved feature extraction and helped the model generalize better compared to classical machine learning approaches.

10. Interactive Demo

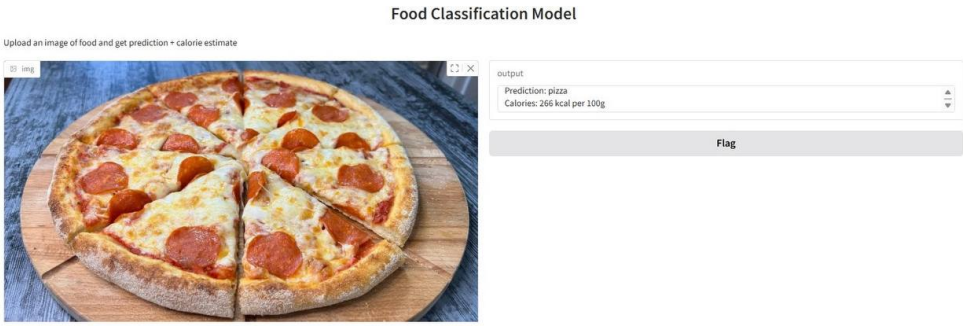

Figure 9: Food Classification Demo Interface

```
def predict(img):
    img = tf.image.resize(img, (224,224))
    img = img / 255.0
    pred = model.predict(img[None])
    return class_names[pred.argmax()]
```

Features:

- Image upload
- Prediction
- Calorie estimation

10.1. Streamlit Application Implementation

To demonstrate the final trained model, an interactive web application was developed using Streamlit. The application enables users to upload an image of food, input its weight in grams, and obtain both the predicted food class and an estimated calorie value.

The trained MobileNetV2 model was loaded using TensorFlow:

```
model = tf.keras.models.load_model("current_model.h5")
```

The application supports 11 food categories, including common dishes such as pizza, sushi, hamburger, and ice cream. A predefined dictionary of calories per 100 grams was used to estimate nutritional values based on model predictions.

Images uploaded by users are resized and preprocessed to match the model input requirements before prediction. The model outputs the predicted class along with a confidence score.

Calorie estimation is performed using a proportional formula based on the predicted class and the user-defined food weight:

```
estimated_calories = (calories_per_100g[predicted_class] / 100) * grams
```

The Streamlit interface displays the following results:

- Predicted food category
- Prediction confidence
- Input food weight (grams)
- Calories per 100 grams

- Estimated total calories

The developed interface provides an intuitive and interactive demonstration of the machine learning model, making it accessible for real-time food recognition and calorie estimation.

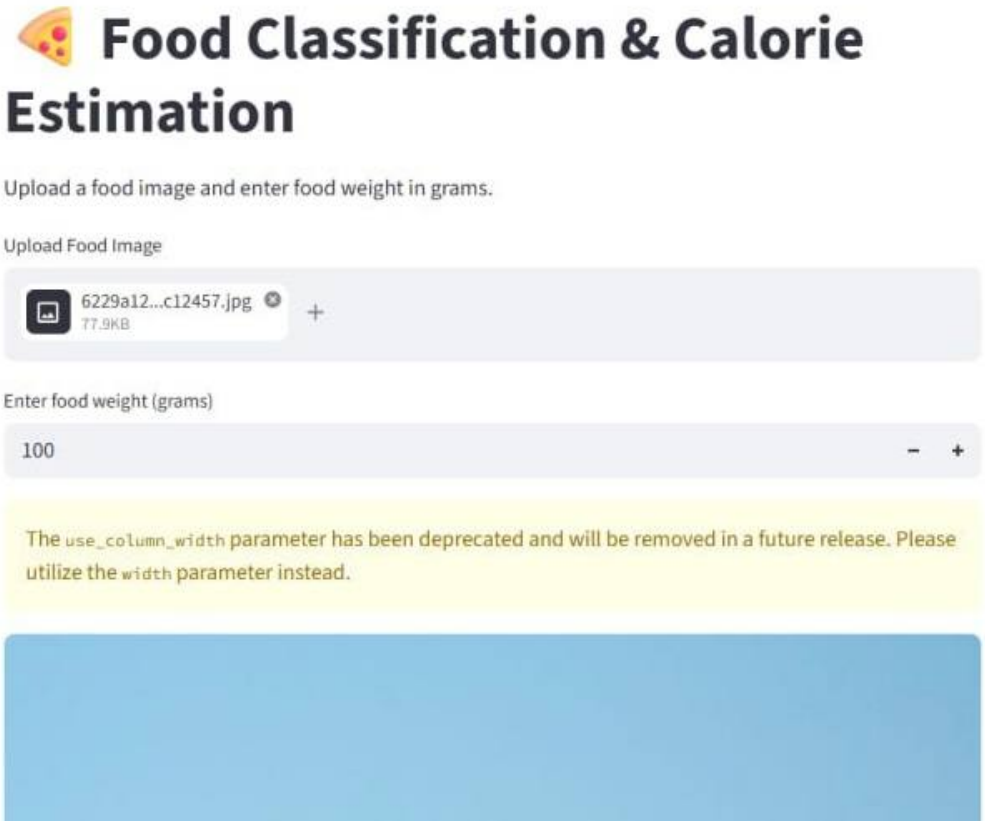

Figure 10: Streamlit application interface - image upload and input section

11. Analysis

Best performing classes:

- Fried rice (F1 = 0.87)
- Chicken curry (F1 = 0.84)

Weakest classes:

- Omelette (F1 = 0.60)
- Hot dog (F1 = 0.63)

Errors are mainly caused by visual similarity between food categories.

The experiments show that transfer learning provides a major improvement over classical machine learning methods for image classification tasks.

12. Conclusion

- MobileNetV2 achieved the best performance (78.8%)
- Transfer learning significantly improves accuracy

- Classical machine learning models are less effective for image data
- Dataset complexity affects model performance
- MobileNetV2 provided the best generalization capability

13. References

- Food11 Dataset (Kaggle)
- MobileNetV2 (CVPR 2018)
- DeepFood (2016)
- Food-101 Dataset (2014)